Hardware accelerators for deep learning (HW2 Report)

Quantize matrix multiplication Assignment

Bashar Abu-Leil | Rami Abu -Mukh

1.Introduction:

Matrix multiplication is a fundamental operation in numerous scientific and machine learning workloads. Its efficient implementation directly impacts performance in modern high-performance computing systems. This assignment focuses on designing and optimizing a custom CUDA kernel for multiplying 32×32 matrices, leveraging hardware acceleration on NVIDIA GPUs — particularly Tensor Cores. The implementation supports two data types: torch.bool and torch.float16, each requiring different strategies to balance computational speed and numerical accuracy.

The primary objective is to approach the minimum of the **average sum of the computation time and the squared L2 error** across a batch of 10,000 randomly generated matrix pairs:

$$\min E[L_2 + computation\ time]$$

To achieve this, we explore a variety of parallelization and quantization techniques, including a warp-level implementation using NVIDIA's WMMA API to utilize Tensor Cores for float16 inputs.

The final implementation is benchmarked on an NVIDIA RTX 2080 Ti GPU, and the results are compared in terms of both efficiency and accuracy. This report outlines the methodology, implementation choices, and performance evaluation of our solution to the quantized matrix multiplication task.

2. Implementation and Methods

We describe the implementation details and methods used for each data type.

2.1 Boolean

2.1.1 First Implementation

The first implementation is a baseline approach that uses shared memory tiling to reduce global memory accesses while computing the boolean matrix multiplication. Each thread block is responsible for computing a 16×16 tile of the output matrix. The kernel loads corresponding tiles of matrices A and B into shared memory, computes partial dot products using logical AND operations, and accumulates the result by checking for any non-zero matches across the tile. The output entry is set to true if at least one match is found.

the kernel implementation:

```
#include <cuda_runtime.h>
#include <device_launch_parameters.h>

#define N 32
#define TILE_SIZE 16

__global__ void matmul_bool_kernel(
    const bool* __restrict__ A,
    const bool* __restrict__ B,
    bool* __restrict__ C,
    int batch_size
) {
    __shared__ bool As[TILE_SIZE][TILE_SIZE];
    __shared__ bool Bs[TILE_SIZE][TILE_SIZE];

    int batch_idx = blockIdx.z;
    int row = blockIdx.y * TILE_SIZE + threadIdx.y;
    int col = blockIdx.x * TILE_SIZE + threadIdx.x;

    if (batch_idx >= batch_size) return;

    int result = 0;
    int matrix_offset = batch_idx * N * N;
```

```
    #pragma unroll
    for (int tile = 0; tile < (N + TILE_SIZE - 1) / TILE_SIZE; tile++) {
        // Load A tile
        int a_col = tile * TILE_SIZE + threadIdx.x;
        if (row < N && a_col < N) {
            As[threadIdx.y][threadIdx.x] = A[matrix_offset + row * N + a_col];
        } else {
            As[threadIdx.y][threadIdx.x] = false;
        }

        // Load B tile
        int b_row = tile * TILE_SIZE + threadIdx.y;
        if (b_row < N && col < N) {
            Bs[threadIdx.y][threadIdx.x] = B[matrix_offset + b_row * N + col];
        } else {
            Bs[threadIdx.y][threadIdx.x] = false;
        }

        __syncthreads();

        // Compute partial dot product for this
        #pragma unroll
        for (int k = 0; k < TILE_SIZE; k++) {
            // Boolean AND + accumulate count
            if (As[threadIdx.y][k] && Bs[k][threadIdx.x]) {
                result++;
            }
        }

        __syncthreads();
    }

    // Store result (any non-zero count becomes true)
    if (row < N && col < N) {
        C[matrix_offset + row * N + col] = (result > 0);
    }
}
```

The results of this implementation:

```
overall mean computation time:  0.06261471986770630   ms
overall mean L1 error:  0.0
overall mean L2 error:  0.0
```

2.1.2 Optimized Boolean Matrix Multiplication via Bit-Packing

To improve the performance of boolean matrix multiplication, we developed a bit-packed kernel that exploits the fixed 32×32 matrix size by compressing each row and column into 32-bit words. This method leverages CUDA's warp-level parallelism and reduces per-element computation from approximately 95 boolean operations to just 2.

Each matrix in the batch is assigned to a single CUDA block. Within a block, 32 threads (one warp) operate in parallel, with each thread responsible for computing one output column of the result matrix. The kernel proceeds in three main stages:

1. Bit-Packing:
    - Each thread packs one row of matrix A and one column of matrix B into 32-bit unsigned integers (uint32_t).
    - This is achieved by iterating over the 32 boolean elements and shifting them into their corresponding bit positions within a single word.
    - This step uses shared memory to store the compressed rows and columns for fast access.
2. Computation:
    - Each thread computes its full output column by performing a bitwise AND between each packed row of A and its own packed column of B.
    - The result of this AND is tested for non-zero, which corresponds to a logical OR over 32 boolean ANDs in the original formulation.
    - This directly implements the boolean semiring operation:

$$\boldsymbol{C}[\boldsymbol{i}][\boldsymbol{j}] = \bigvee_{\boldsymbol{k}=\boldsymbol{0}}^{\boldsymbol{31}} \boldsymbol{A}[\boldsymbol{i}][\boldsymbol{k}] \ \wedge \boldsymbol{B}[\boldsymbol{k}][\boldsymbol{j}]$$

3. Result Storage:
    - The boolean result is written back to the output matrix C, completing one full matrix in just a few warp-level operations.

This optimization is particularly effective due to the alignment between the 32×32 matrix size and the warp size (32 threads), allowing all output columns to be processed simultaneously within a single warp. Furthermore, by transforming logical operations into bitwise operations, memory bandwidth usage and instruction count are significantly reduced.

The resulting kernel is highly efficient and well-suited for large batch sizes, achieving substantial speedup over the naive implementation while preserving semantic correctness.

The kernel implementation:

```
#include <cuda_runtime.h>
#include <device_launch_parameters.h>
#include <stdint.h>

#define N 32
#define THREADS_PER_BLOCK 32

__global__ void matmul_bool_kernel_bitpacked(const bool* __restrict__ A,
                                             const bool* __restrict__ B,
                                             bool*       __restrict__ C,
                                             int                      b
atch_size)
{
    const int batch_idx = blockIdx.x;
    if (batch_idx >= batch_size) return;

    const int tid = threadIdx.x;
    __shared__ uint32_t A_rows[N];
    __shared__ uint32_t B_cols[N];

    const int mat_off = batch_idx * N * N;


    uint32_t row_word = 0u;
    #pragma unroll
    for (int k = 0; k < N; ++k)
    {
        bool bit = A[mat_off + tid * N + k];
        row_word |= (uint32_t)bit << k;
    }
    A_rows[tid] = row_word;

    uint32_t col_word = 0u;
    #pragma unroll
    for (int k = 0; k < N; ++k)
    {
        bool bit = B[mat_off + k * N + tid];
        col_word |= (uint32_t)bit << k;
    }
```

```
    B_cols[tid] = col_word;

    __syncthreads();


    for (int row = 0; row < N; ++row)
    {
        uint32_t mask = A_rows[row] & B_cols[tid];
        bool     val  = (mask != 0u);
        C[mat_off + row * N + tid] = val;
    }
}
```

The result of this implementation:

```
overall mean computation time:  0.0146691201627254 48   ms
overall mean L1 error:  0.0
overall mean L2 error:  0.0
```

2.1.3 Two-Warp Ballot-Based Boolean Matrix Multiplication

This kernel optimizes the computation of Boolean matrix multiplication by using warp-level primitives and bitwise logic. Each warp of 32 threads is assigned to compute the multiplication of a single 32×32 Boolean matrix, where each thread is responsible for computing one output column. By grouping two warps in a single thread block, the launch overhead is amortized across two matrix computations, thereby improving GPU occupancy.

The kernel works by compressing the data into bitmasks:

- Each thread first packs its assigned column of matrix B into a 32-bit integer (col_mask), where each bit represents a Boolean value from the column.
- For every row in matrix A, a warp-wide ballot operation (__ballot_sync) is used to construct the corresponding 32-bit row mask. This gathers the active thread bits across the warp into a single integer.
- The kernel then computes a bitwise AND between the row mask and the column mask. If any bit is set (i.e., the result is non-zero), the corresponding output element in matrix C is set to true.

The computation for each entry, $C[i][j]$, remains the same as the previous implementation.

The kernel implementation:

```
#include <cuda_runtime.h>
#include <device_launch_parameters.h>
#include <stdint.h>

#define N                32
#define WARP_SIZE        32
#define WARPS_PER_BLOCK   2
#define FULL_MASK   0xFFFFFFFFu


__global__ void matmul_bool_kernel_2warp(const bool* __restrict__ A,
                                         const bool* __restrict__ B,
                                         bool*       __restrict__ C,
                                         int                      batch
_size)
{

    const int warp_idx = threadIdx.x >> 5;
    const int lane     = threadIdx.x & 31;

    const int matrix_id = blockIdx.x * WARPS_PER_BLOCK + warp_idx;
    if (matrix_id >= batch_size) return;
    const int col      = lane;
    const int base     = matrix_id * N * N;


    uint32_t col_mask = 0u;
    #pragma unroll
    for (int row = 0; row < N; ++row) {
        bool bit   = B[base + row * N + col];
        col_mask  |= (uint32_t)bit << row;
    }
    #pragma unroll
    for (int row = 0; row < N; ++row) {
        bool     a_bit   = A[base + row * N + col];
        uint32_t rowmask = __ballot_sync(FULL_MASK, a_bit);
        bool     val     = (rowmask & col_mask) != 0u;
        C[base + row * N + col] = val;
    }
}
```

The results of this implementation:

```
overall mean computation time:  0.008493440076708794   ms
overall mean L1 error:  0.0
overall mean L2 error:  0.0
```

2.1.4 Wrapper Modification of Kernel Initiator:

While the kernel itself remained unchanged, we modified the wrapper function responsible for launching it. Specifically, we allocated the result tensor **statically**. This change resulted in an average computation time of **0.007** ms.

illustration of the changes:

```
torch::Tensor matmul_cpp(torch::Tensor A, torch::Tensor B)
{
    const int batch = A.size(0);
    static torch::Tensor C = torch::empty_like(A);

    if (A.scalar_type() == torch::kBool) {
        matmul_bool_cuda(A.data_ptr<bool>(),
                         B.data_ptr<bool>(),
                         C.data_ptr<bool>(),
                         batch);
    } else if (A.scalar_type() == torch::kHalf) {
        matmul_fp16_cuda(reinterpret_cast<__half*>(A.data_ptr<at::Half>()),
                         reinterpret_cast<__half*>(B.data_ptr<at::Half>()),
                         reinterpret_cast<__half*>(C.data_ptr<at::Half>()),
                         batch);}
    return C;
}
```

The results of this implementation :

```
Printing Summary over runs:
Input type was set as a single torch Tensor
overall mean computation time:  0.007477491262555123   ms
overall mean L1 error:  0.0
overall mean L2 error:  0.0
```

2.2 float:

2.2.1 HW1 implementation:

By modifying the data type to fp16 in our HW1 kernel implementation, we achieved an average computation time of **0.06 ms** with **zero numerical error**.

2.2.2 Tensor Core-Based Matrix Multiplication (FP16)

This kernel leverages NVIDIA's Tensor Cores through the WMMA API to accelerate the multiplication of 32×32 matrices with half-precision inputs. Each matrix is divided into four 16×16 subtiles, and each subtile is computed by a single warp (32 threads), resulting in four blocks per matrix.

The computation is split into two phases, each handling one half of the inner dimension. For each phase, 16×16 tiles from the input matrices are loaded, multiplied using Tensor Core operations, and accumulated into a temporary result. This result is stored in shared memory and then written back to global memory after converting it to half precision.

This implementation is designed to take full advantage of Tensor Core performance while preserving acceptable numerical accuracy.

the kernel implementation:

```
#include <cuda_runtime.h>
#include <cuda_fp16.h>
#include <mma.h>
using namespace nvcuda::wmma;

static constexpr int MATRIX_SIZE = 32;
static constexpr int TILE        = 16;
static constexpr int WARPSIZE    = 32;

__shared__ float Ctemp_shared[TILE * TILE];

extern "C" __global__ void matmul_kernel_32x32_wmma_fp16_fp32(
    const __half* __restrict__ A,
    const __half* __restrict__ B,
    __half*       __restrict__ C)
{
    int global_block_id = blockIdx.x;
    int matrix_id      = global_block_id >> 2;
    int subtile_id     = global_block_id & 3;
```

```
    int sub_i = (subtile_id >> 1) & 1;
    int sub_j = (subtile_id >> 0) & 1;

    const __half* Aptr = A + matrix_id * (MATRIX_SIZE * MATRIX_SIZE);
    const __half* Bptr = B + matrix_id * (MATRIX_SIZE * MATRIX_SIZE);
    __half*       Cptr = C + matrix_id * (MATRIX_SIZE * MATRIX_SIZE);

    int laneId = threadIdx.x;

    fragment<accumulator, TILE, TILE, TILE, float> acc_frag;
    fill_fragment(acc_frag, 0.0f);


    {
        fragment<matrix_a, TILE, TILE, TILE, __half, row_major> a_frag;
        fragment<matrix_b, TILE, TILE, TILE, __half, row_major> b_frag;

        const __half* Atile0 = Aptr + (sub_i * TILE) * MATRIX_SIZE + 0;
        const __half* Btile0 = Bptr + 0 * MATRIX_SIZE + (sub_j * TILE);

        load_matrix_sync(a_frag, Atile0, MATRIX_SIZE);
        load_matrix_sync(b_frag, Btile0, MATRIX_SIZE);

        mma_sync(acc_frag, a_frag, b_frag, acc_frag);
    }


    {
        fragment<matrix_a, TILE, TILE, TILE, __half, row_major> a_frag;
        fragment<matrix_b, TILE, TILE, TILE, __half, row_major> b_frag;

        const __half* Atile1 = Aptr + (sub_i * TILE) * MATRIX_SIZE +
TILE;
        const __half* Btile1 = Bptr + TILE * MATRIX_SIZE + (sub_j *
TILE);

        load_matrix_sync(a_frag, Atile1, MATRIX_SIZE);
        load_matrix_sync(b_frag, Btile1, MATRIX_SIZE);

        mma_sync(acc_frag, a_frag, b_frag, acc_frag);
    }


    store_matrix_sync(
        Ctemp_shared,
```

```
        acc_frag,
        TILE,
        mem_row_major
    );


    __syncthreads();


    {
        for (int e = 0; e < 8; ++e) {
            int localIndex = laneId + e * WARPSIZE;
            int r_local    = localIndex / TILE;
            int c_local    = localIndex % TILE;

            int r_global = sub_i * TILE + r_local;
            int c_global = sub_j * TILE + c_local;

            int write_idx = r_global * MATRIX_SIZE + c_global;
            float val32 = Ctemp_shared[r_local * TILE + c_local];
            Cptr[write_idx] = __float2half(val32);
        }
    }

    __syncthreads();
}
```

The results of this implementation:

```
overall mean computation time:  0.0413839998841285 74   ms
overall mean L1 error:  0.0
overall mean L2 error:  0.0
```

2.2.3 Wrapper Modification of Kernel Initiator:

While the kernel itself remained unchanged, we modified the wrapper function responsible for launching it. Specifically, we allocated the result tensor **statically**. This change resulted in an average computation time of 0.013 ms.

illustration of the changes:

```
torch::Tensor matmul_cpp(torch::Tensor A, torch::Tensor B)
{
    const int batch = A.size(0);
    static torch::Tensor C = torch::empty_like(A);

    if (A.scalar_type() == torch::kBool) {
        matmul_bool_cuda(A.data_ptr<bool>(),
                         B.data_ptr<bool>(),
                         C.data_ptr<bool>(),
                         batch);
    } else if (A.scalar_type() == torch::kHalf) {
        matmul_fp16_cuda(reinterpret_cast<__half*>(A.data_ptr<at::Half>()),
                         reinterpret_cast<__half*>(B.data_ptr<at::Half>()),
                         reinterpret_cast<__half*>(C.data_ptr<at::Half>()),
                         batch);}
    return C;
}
```

The results of this implementation:

```
Printing Summary over runs:
Input type was set as a single torch Tensor
overall mean computation time:  0.013298444804549217   ms
overall mean L1 error:  0.0
overall mean L2 error:  0.0
```

3. Results and discussion:

Using float16 for both inputs, we achieved an average computation time of **0.013**ms. Using bool for both inputs, we achieved an average computation time of **0.007**ms. Quantization did not provide any notable performance gain due to the small batch size used in our tests.